

60.5. La fonction `cacherPanneau()` — ôter le panneau

`#include <cacherPanneau>` : `cacherPanneau()` ôte le panneau de l'écran, sans rien perdre de ce qui y est écrit.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `cacherPanneau()` **éteint** le panneau. Ce qui y est écrit reste en mémoire : un `panneau(x, y)` le remontre tel quel.

La ligne à écrire : `#include <cacherPanneau>`.

Pas d'argument.

Ce qu'elle coûte : environ **75 octets**, avec la mise en place de l'écran.

Essaie : remontre-le à l'image 240 avec `panneau(0, 128);`.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **337 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage	– la mise en route de la console
main()	46 o	← démarrage	– ta fonction
VBlank	8 o	← la console, à chaque image	– compte les images, 60 fois par seconde
AttendreImage	28 o	← image()	– attend l'image suivante
AttendreVBlank	16 o	← démarrage	– attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte	– attend de pouvoir écrire une case
EcrireTexte	10 o	← textePanneau()	– écrit un texte, lettre par lettre
CopierTuiles	67 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
Tuiles	80 o	← CopierTuiles	– 5 lettres sur 43
"MESSAGE"	7 o	← textePanneau()	– un texte gravé

Lettres gravées : AEGMS

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

panneau()	× 1	15 o sur place, dans main()
textePanneau()	× 1	11 o sur place, dans main() + EcrireTexte (10 o)
cacherPanneau()	× 1	6 o sur place, dans main()
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)
images()	× 1	2 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

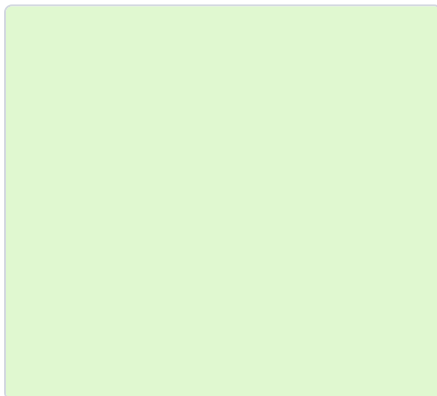
```
// ---- #include <cachePanneau> : ôter le panneau ----  
// Le panneau se montre au départ ; à l'image 120 (deux secondes), il part.  
  
#include <textePanneau>  
#include <panneau>  
#include <cachePanneau> // cachePanneau() : l'ôter, sans l'effacer
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    textePanneau(1, 0, "MESSAGE");  
    panneau(0, 128);  
  
    while (true) {  
        image();  
        if (images() == 120) { // images() compte les images depuis l'allumage  
            cachePanneau();  
        }  
    }  
}
```

3. Ce qui se passe, étape par étape

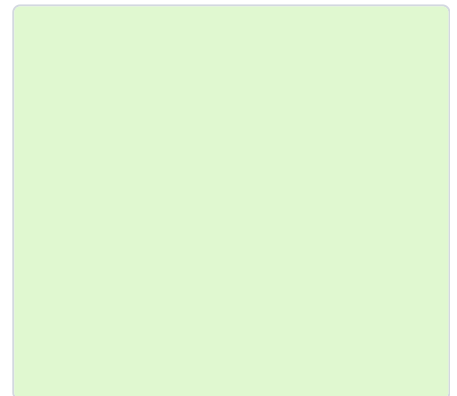
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout de trois secondes — image 180

4. Ce qu'on doit voir

MESSAGE en bas de l'écran, qui disparaît au bout de deux secondes.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au départ, le panneau est là
- ✓ deux secondes plus tard, il est caché

✓ son texte n'est pas effacé

Ensuite : **60.6. La fonction paletteFond() — les quatre nuances du fond** — chapitre 9 : Ranger ce qu'on manipule. · gameboy3
— écrire une cartouche Game Boy en C++